

Voice-Native Rule Execution on Smartphones: Evaluating Small Language Models for On-Device Automation

Achindra Sharma

PG Scholar, Department of Computer Science

CHRIST (Deemed to be University)

Bangalore, India

achindra.sharma@mca.christuniversity.in

Abstract—Voice interaction has become the normal interaction method between people and their gadgets in such a manner that it is more or less close to human communication. The adoption of these systems however is still a hard task because of privacy issues and latency problems since they require a lot of cloud processing. In overcoming this challenge, there has been a recent research of ensuring that capable voice systems can be installed to perform directly on the edge and mobile devices. Advances in speech recognition and smaller and more efficient language models have demonstrated evident advances in the overcoming of these constraints. Nonetheless, the majority of available researches focus on these parts individually and little research correlates the two domains with each other to assess the extent of inputs in speech to real, well-structured and executable actions. It is in this gap that this paper discusses a comprehensive voice-native framework which combines quantized speech recognition with Small Language Models on a standard Android smartphone. The study evaluates 5 different model architectures on 600 test cases to find out the best tradeoff between reasoning depth and responsiveness, 2-billion parameter models with 83.3 percent accuracy and 421ms latency attain the best tradeoff. These findings identify that this kind of systems has Semantic Resilience, and they have the capability of correcting the transcription mistakes and ensuring high reliability. These results demonstrate that high quality and private automation do not demand server-grade infrastructure.

Keywords—On-device language modelling, Mobile voice interfaces, Speech-to-rule translation, Edge-based automation, Real-time command execution.

I. INTRODUCTION

Large Language Models (LLMs) have revolutionized artificial intelligence by providing some advanced reasoning, generation, and communication capabilities in various applications of day-to-day life. However, these systems have high energy consumption, dependence on the internet, and serious user privacy issues among others due to their intensive computing and memory requirements. Researchers are in turn responding to this by developing Small Language Models (SLMs) that exhibit strong reasoning behaviors using less resources. They are small-scale models characterized not only by size but also by the ability to provide contextually rich intelligence within the constraints of a very strict set of hardware requirements. SLMs are a movement toward an architecture-based, data-

centric training paradigm of sustainable, on-device intelligence, independent of massive server-grade infrastructure [1].

As part of these trends, edge computing innovations have been used to support a new generation of models to run directly on the device. Modern mobile processors and neural units can now make real-time decisions through software and hardware optimizations and this is critical in integrating smart systems into everyday appliances that are not permanently connected to the cloud. However, the application of these models on mobile devices leads to additional limitations such as memory and battery life and temperature variability, which cause inference as a persistent problem. The trade-offs required to make it accurate without compromising the performance of the hardware gives the reliability of operation a challenge [2].

Voice interaction today has emerged as the natural way of people interacting with their devices giving users a close to human interaction experience. Nevertheless, the implementation of this type of system is not simple because of privacy concerns and delays caused by the transfer of voice data to the cloud to be processed. To fight this, recent researches have tried to empower efficient voice systems to serve as direct interfaces on edge and mobile devices. Speech recognition progress has also shown clear progress in the direction of preventing these limitations by downsizing the large cloud models to accommodate various accents and noisy environments in offline situations [3]. Nevertheless, these components are analyzed individually in the majority of the current studies. Limited literature exists to directly correlate speech recognition with SLMs to determine the degree of effectiveness of speech inputs in being translated into formal, executable actions on consumer equipment.

In order to fill this gap, this paper describes a fully voice-native architecture that combines support of quantized speech recognition with SLMs on a regular Android smartphone. Contrary to the conventional methods, which imply the use of cloud APIs, this system comprises a pipeline that combines Whisper Tiny architecture based on transcription, and 4-bit quantized SLMs based on logic implementation. The framework converts natural language input to strict JSON payloads

that can control native Android APIs (e.g. the flashlight, the volume and the screen brightness) without any data leaving the device by using a Chain-of-Thought reasoning protocol.

Based on this framework, a set of 5 different model architectures (Qwen, SmoLLM, Gemma, Llama, and Phi) are evaluated on 600 unique test cases to understand the practicability of on-device automation. We have found that the level of 2-billion parameter scale is the critical point in terms of inflection of mobile hardware, providing the most efficient tradeoff between computational latency (421ms) and reasoning accuracy (83.3%). In addition to crude measures of performance, this paper describes the fact that there is a Semantic Resilience effect, i.e. the ability of the language model to serve as a corrective layer in the face of phonetic errors created during the transcription. These results demonstrate that with good quality, privately implemented automation on mid-range consumer hardware can be implemented without incurring server grade infrastructure.

II. LITERATURE REVIEW

One of the main issues with transferring artificial intelligence off of cloud servers to personal devices is the decoupling of high-level reasoning and massive numbers of parameters. Although the common wisdom dictated that the ability of a model was directly proportional to its size, recent studies have shown that the intelligence of the model is more strongly influenced by the quality of the training data and the efficiency of its internal structure. This trade-off is imperative in the mobile setting of strictly constrained memory and power. To overcome these limitations, two main courses have been taken by the field: data-centric development and architectural optimization.

To gain the benefits of model development without scaling, scholars have moved to data-centric development. An SLM, SmoLLM2, suggested by Ben Allal et al. suggests that capability is not more related to the model size, but rather to the quality of the data. This approach highlights the fact that the quantities of web data are typically noisy and redundant and it results in inefficient models that use up computational resources without need. SmoLLM2 project found that with a training set of made-up and textbooks-like examples smaller and filtered, a model of less than two billion trainable parameters could learn to perform complex tasks like mathematics and logic, without using additional data. The direct implications of this discovery to on-device AI are that it is possible to train much better models to be highly competent not by scale [4].

Along with the data enhancement, further improvement of re-engineering model architectures to mobile hardware limits has been even more successful. MobileLLM was the architecture proposed by Liu et al. which was specifically designed to be compatible with the mobile processors. MobileLLM is built in deep and thin mode as opposed to the server-grade models that are organized in a wide and shallow way. This architecture minimizes the amount of data that is required to be sent between levels of memory (e.g. DRAM and on-chip

caches) which is typically the largest size constraint to speed and power consumption of a mobile device. This demonstrated that even very small models (at the level of 350 million parameters) could be trained at least with an architecture that had been co-designed with the hardware, and could maintain coherent and long-form conversations previously thought to be beyond the scope of local hardware [5].

In spite of the fact that the optimization of existing architectures is an efficient solution is given, certain studies show that this Transformer is not efficient in certain edge cases. Gu and Dao have broken this dominance with a selective state space model, Mamba. Performance of traditional Transformer decreases as the conversation length increases because the traditional model must re-run the entire history each time a new token occurs. Mamba on the other hand puts a condensed version of this history in a small dynamic state. It allows it to think in a linear manner and therefore future voice assistants will have the potential of supporting very long contexts without the geometric scaling up of slowdown and battery usage of attention based models [6].

A recent technological development in the area is parameter-efficient fine-tuning (PEFT). Low-Rank Adaptation (LoRA) proposed by Hu et al. is a model architecture that modifies model adaptation to some tasks. LoRA does not resort to the computationally demanding task of updating model parameters by simply freezing the original model and learning on top of it the specific task-related differences in a small, trainable set of adapting layers. According to this, Dettmers et al. suggested QLoRA that trains LoRA adapters alongside quantizing the base model. The technology allows personalization of the computational aspect on-device and a general voice assistant is enhanced so as to learn the vocabulary or habits of a specific user without the need to retrain the entire model [7], [8].

Along with model adaptation, it is important to effectively run them on-device. To address this latency of autoregressive generation, Leviathan et al. suggested Speculative Decoding. This technique uses a smaller and faster model of drafter to produce candidate words, which are verified in one step by the larger and more accurate model. This strategy will reduce the load on the memory hardware considerably and make the AI more responsive and responsive to the user of this system much more natural to the user [9].

Finally, the practical functionality of on-device assistant is the ability to perform and not only speak. The current studies have therefore been focusing on transforming language models into agents that can interact with other software. In order to bridge the natural language and technical implementation gap, Qin et al. devised of the ToolLLaMA framework. They showed that with proper training, a language model can reason and comprehend thousands of different software interfaces (APIs), and effectively make the model a universal translator of human instructions in essence [10]. Nevertheless, the ability to execute code poses a tremendous reliability risk. To address this issue, Patil et al. proposed Gorilla, a fine-tuning system that makes models adhere to a valid API syntax and tools available to the letter. Their results showed that basing the

response of the model on this basis, the invalid errors in calling functions might be significantly reduced, and a system of stable, reliable automation could be implemented in this way [11].

The connection between the ASR and the SLM is supposed to be fluent in order to experience voice-first. Qin et al. proposed Tiny-Align, which is a system that seeks to bridge the gap in speech recognition and in language understanding in an efficient manner. Tiny-Align does not need to convert the speech to text followed by text to logic; it simply transforms the internal representations of the speech model to the language model. This method is far quicker and more interactive than a conventional command and control interface, and a more natural interaction is possible in this way [12].

With these agentic systems turning into a practice rather than a skill, they are now concerned with sustainability and reliability. MobileAIBench, offered by Wang et al., resolved the lack of on-device AI-based standardized testing. Their work concluded that in order to make an assistant a useful, everyday device, it is necessary to pay attention not only to its cleverness but also to the cost of its thinking in terms of energy utilization and thermal feasibility. Laskaridis et al. in their MELTing Point study found that the performance of a device may decrease to ineffectiveness after just a few minutes of continuous use because of overheating of the device, which supports this holistic perspective on the matter at hand, as well as prompts reflection on the futility of excessive dependence on electronics in everyday life and the failure to realize that some tasks can be performed without technology at all, even though this very technology is essential in other situations [13], [14]. One must also examine the quality of reasoning on a case by case basis, in addition to physical performance. In reply to this, Liu et al. suggested an evaluation system, AgentBench, that exposes models to a wide range of multi-step problems based on real-world tasks. Such thorough testing demonstrated that a large fraction of models that are good at simple conversation have difficulty when asked to plan multiple steps ahead or maintain context in a long conversation, which can be used to identify a method by which models that can simulate language and models which have a true agentic ability to reason can be separated through experimentation [15].

Altogether, the literature under analysis presents a promising trend. The research community has been able to come up with efficient architectures, improved compression algorithms and strongly built frameworks to execute what can be done with the device. However, there is still a huge gap. By far most of the literature analyzes these aspects separately, such as, optimizing quantization without testing agentic reasoning, or by creating sophisticated agents, yet one that continues to use cloud-based hardware. The empirical literature on combining these disparate components into a single, end-to-end voice automation pipeline and testing these components as a coherent whole is in notable deficit.

III. MATERIALS AND METHODS

Voice-based automation has been informed by two architectural paradigms both cloud-based processing and the other rigid on-device signal processing. Although the two approaches have facilitated the opening up of voice interfaces, both approaches have their structural disadvantage that compromise the production of a holistic and private, responsive and intelligent mobile agent. This is how the commercial voice assistant systems in the industry move into a new direction where the mobile phone becomes more of a relay point of data. Only the local site hardware in the workflow will accomplish the task of audio compression and wake-word detection. Upon activation, the raw audio signal is then transmitted out over a standard web protocol to one centralized data center where it is processed with the massive LLMs, which usually contain hundreds of billions of parameters, to execute the computationally expensive task of Automatic Speech Recognition (ASR) and Natural Language Understanding. This architecture allows the state of the art models to be used without overloading of the mobile processor being necessary, but it has two bottlenecks that the architecture requires. The system will be at first conditionally dependent on the stability of the network as a whole, and the latency of an interaction not being determined by the rate of inference, but the round-trip latency of a network packet, and thus causing the assistant to slow down in areas with bad connectivity. Second, the architecture will entail the data egress that implies that voice files holding the background biometric and potentially sensitive information about the user need to traverse the boundary of the machine, and it inescapably results to the introduction of the risk to privacy.

The older variants of the mobile assistants were made to include the slot filling and finite state grammars to overcome the latency and privacy issues that were experienced with the cloud systems by offline command recognition. Lightweight acoustic models, such as Convolutional Neural Networks (CNN) models, are typically involved in the inputs of such a system to control audio functionality to a pre-existing group of command templates. However, excessive lack of semantic argument frames the approach. Neither is it a system, which operates on the principle of interpretation of languages, the system simply applies the rigid pattern recognition whereby when the user fails to deliver the necessary grammar pattern as in, which says that the surrounding is too bright rather than telling the computer to set the brightness to zero, the system cannot understand the command. They lack the knowledge of the world to give intent to the context hence; they experience brittle user experiences where the user must recall some phrases to use with the device.

A. Integrated Speech-to-Logic Architecture

To address the latency problem of cloud-reliant systems and the semantic rigidity of the speech to logic interfaces of the early 21st century, one had to develop one, on-the-box Speech-to-Logic architecture. The prototype is developed as a native Android application, which has been developed using Flutter

cross-platform framework. More importantly, Cactus SDK was included to power the complex lifecycle of LLMs directly onto mobile devices.

This philosophy of architecture is what is known as Privacy by Default. The entire computational unit including the recording of the acoustic vibrations and executing system code in this paradigm is achieved on the machine, in the CPU. The structure is structured in a manner that will at no time touch the device border with any data, be it voice records or semantic logs, thereby eliminating privacy risks of processing stored data at the server.

B. The Three-Phase Serial Execution Pipeline

The proposed system is realized within a three-stage pipeline that employs a serialization mechanism to minimize the memory contention and achieve the stability of the inference on resource-constrained systems.

1) *Acoustic Perception*: The former converts unstructured audio in raw format into a high-fidelity text. The device microphone is used to record audio at 16 kHz to the system, and is deciphered by Whisper Tiny, an ASR model in the shape of a transformer. The transformer architecture guarantees the integrity of transcription in noisy acoustic context as compared to the conventional CNN acoustic models, which are generally susceptible to noise. The model weights were scaled to integers in 8-bit format in a manner that optimally permits real time performance. This is because this capability is enough to have the system cut its cost of computation per matrix multiplication by enabling the system to transcribe up to 10 seconds of audio at virtually no latency on a common mobile CPU.

2) *The Reasoning Engine*: The core reasoning engine, which has the SLM, is then fed with the transcribed text. A modular backend was implemented to test empirically the trade-off between the size of models and their reasoning ability; this could swap different model structures hot. The paper has compared benchmarking of five parameters models of 0.6 billion to 3.8 billion, such as Qwen, SmoLLM, Gemma, Llama and Phi. The Gemma family of models, such as, are initially architected with on-device execution in mind, including architectural optimizations and quantization-aware training that way any can remain able to run it even when it has been heavily compressed [16].

3) *Execution and Control*: The final step entails translation of the abstract thinking of the model into action. The application layer will read the organised output of the model and compare the keys present with the native android API endpoints. This is a self-managed hardware interface which is a flexible execution layer that can also manage hardware interfaces like the flashlight, volume controller, and the screen brightness without having a human operator.

C. Memory Optimization and Quantization

The most significant technical issue of using LLMs on consumer smartphones is the lack of RAM which has been widely discussed in surveys of model compression [17]. The model uses 6GB of RAM on the average and it is a full

precision (FP16) 3-billion parameter model. This load would definitely cause an 8 GB device using the Android operating system to crash with the overhead.

All the model architectures in this work were converted to the GGUF format with 4-bit quantization to fix this bottleneck. This method of compression yields a memory footprint of 0.5 to 0.7GB per billion parameters. Such degree of compression is enabled by the state-of-the-art post-training quantization methods. One approach is GPTQ, where quantization is optimized layer by layer. It is a mathematical method that determines the way to change the remaining weights in a model to compensate the accuracy error that quantizing other weights caused, making it possible to perform aggressive compression with minimal accuracy loss [18]. Another and much more efficient approach is AWQ (Activation-aware Weight Quantization) that is grounded on another principle: all weights are not created equal. AWQ notes that the weights that process most important and big input signals (activations) are the most important to reasoning. It thus safeguards this little yet important portion of the model brain and aggressively compacting the others, maintaining performance on demanding tasks [19]. Reminding the memory bandwidth of half-precision formats, the biggest models (3.8B parameters) were able to be set in memory with the operating system, meaning that they were stable in inference [19].

The method is further complemented by optimizations at the system level which allow the devices to execute even larger models than their available RAM. Alizadeh et al. suggest the technique of "LLM in a Flash" that considers the phone flash storage as an expansion of the phone memory. It loads only those parts of the model into RAM on-demand thus not loading the whole model at the time they are not being needed in computation rather than loading them all at once like in the traditional method of loading a complete model before it is ever needed in computation [20].

D. Chain-of-Thought Protocol

The natural language is prone to ambiguity whereas programming interfaces demand rigid syntax. In order to overcome this loophole, a Chain-of-Thought (CoT) system prompt was proposed as a cognitive governor. The low temperature value was kept at 0.1 in order to obtain deterministic results. Performance of the system is also improved by being inspired by the current on-device execution frameworks. As an example, the pipeline has the advantage of controlling computation within heterogeneous processors in a device, which has been investigated in systems such as PowerInfer [21]. Besides, to enhance responsiveness, the design of the system integrates concepts of speculative decoding wherein fast but smaller model can write some responses that are later authenticated by the main model, a technology that lies at the heart of systems such as LLMcad [22]. The protocol takes the model through a six step thinking process:

The protocol is as follows:

1) *Intent Isolation*: The model identifies the core verb and objective, filtering out conversational noise.

- 2) *Synonym Management*: It maps user vocabulary (e.g., “hush”) to rigid system nomenclature (e.g., “Do Not Disturb”).
- 3) *Parameter Derivation*: The logic engine distinguishes between absolute values (integers) and relative adjustments (increments/decrements).
- 4) *Temporal Resolution*: A key innovation of this framework is the translation of abstract time expressions, such as “in the next hour,” into absolute integer values that can be registered as OS timers.
- 5) *Context Mapping*: The model infers implicit settings from environmental indicators; for instance, deducing that a “sleeping” context implies disabling the screen or radio.
- 6) *JSON Encoding*: Finally, the model justifies the selected action and encodes the result into a strict JSON payload for the application layer.

E. Experimental Setup and Evaluation Methodology

To test the proposed framework, a strict test environment was developed to recreate the stringent conditions of the mobile usage in real world situations and yet come up with a laboratory quality of consistency. All the experiments in benchmarking were conducted on a standard hardware platform, i.e., the OPPO F25 Pro which was selected to represent the mid-range consumer smart phone market. It has been powered by a MediaTek Dimensity 7050 System-on-Chip (SoC) that has an octa-core CPU architecture, which has balanced power consumption and performance, 8GB of LPDDR4X RAM.

To make the CPU shed the raw inference capabilities, any hardware acceleration capabilities like GPU offloading and the Neural Processing Unit (NPU) being explicitly disabled. This constraint provided a worst-case performance base, which ensures that the performance will be applicable to cheaper devices that do not have dedicated AI accelerators. Moreover, temperature was maintained at its maximum levels as this helps to minimize variance: there was a 30-second delay between the iterative batches to minimize thermal throttling and prevent scaling of the operating system with frequency.

TABLE I
DATASET COMMAND COMPLEXITY TAXONOMY

Category	Reasoning Requirement	Example Utterance
Direct Commands	Explicit Verb-Object mapping. Low ambiguity.	“Turn off flashlight”
Contextual Queries	Inferring system settings from environmental descriptions.	“Make it suitable for sleeping”
Parameter Instructions	Extraction of precise integer or boolean values from text.	“Set volume to 45 percent”
Temporal Logic	Resolving abstract time expressions into absolute OS timers.	“Quiet mode for the next hour”
Rule-Based Triggers	Constructing conditional (If-Then) automation routines.	“Silence notifications when I am driving”

The test process was automated with a specially developed benchmarking service that is a part of the application layer directly. This service took care of the injection of a pruned dataset of 120 different voice commands, which are divided into five different complexity levels as listed in Table I. To make the evaluation close to the unpredictability of acoustic situations, the system did not analyze perfect text inputs. Instead the stochastic simulation layer was added to add synthetic noise and phonetic errors to the command strings and produced a dataset with a Mean Word Error Rate (WER) of 0.129. Such an approach made it possible to measure the Semantic Resilience, which is the ability of the SLMs to make the proper inference of the user intent despite the severe mistakes in their transcription. During the testing, the models have been parameterized to have a temperature of 0.1 which has the effect of eliminating the randomness of generation to provide deterministic and reproducible output patterns.

Four key measures were used to measure quantitative performance across 600 total test cases, including Success Rate which measures overall correctness of the execution, Function Accuracy which measures how accurately the tool is selected, Parameter Accuracy which measures how well the system extracts numerical arguments and End-to-End Latency which measures the total wait time as perceived by the user. These results were validated by a strict statistical framework with the help of the scipy and pandas libraries of Python. A Chi-square test of independence ($\chi^2 = 2.06$, $p = 0.72$) was used to verify the uniformity of model performance across the various categories of command and a one-way Analysis of Variability (ANOVA) ($F = 3.13$, $p = 0.014$) was used to indicate that the size of the model was statistically significant in determining the success of automation. Also, the rank correlation as developed by Spearman was used to measure the association between the rates of acoustic errors and the success of execution ($\rho = -0.114$, $p = 0.005$) that gives a solid statistical foundation to the deployment recommendations.

IV. RESULT AND ANALYSIS

The test performance was done on 600 special test cases where the performance of five various SLMs on the OPPO F25 Pro was tested. The trade-off that was critical to measure between model size, operational efficiency and semantic robustness under simulated acoustic conditions was not its ultimate goal but a benchmark that it intended to measure.

The results provided in Table II as the summary of the aggregate measures of performance indicated that there is a clear hierarchy of model capability where the number of parameters is dominant. The most successful, in total percentage of 85.8-percent, architecture was the Llama 3.2 (3.0B) which is more efficient to execute complex and multi-step instructions. It was succeeding Phi 3.5 Mini (84.2 percent), and Gemma 2 (83.3 percent). However, Qwen 3 (0.6B) was the least successful model with only 70.0 percent success rate. Nevertheless, these indicators of accuracy do not give the full picture of the mobile viability, it needs to be put in perspective with speed

TABLE II
AGGREGATE MODEL PERFORMANCE METRICS

Model	Parameters	Success Rate	Function Accuracy	Parameter Accuracy	Mean Latency
Qwen 3	0.6B	70.0%	81.7%	70.0%	178ms
SmolLM	1.7B	79.2%	87.5%	79.2%	347ms
Gemma 2	2.0B	83.3%	89.2%	83.3%	421ms
Llama 3.2	3.0B	85.8%	90.8%	84.2%	554ms
Phi 3.5	3.8B	84.2%	90.0%	83.3%	652ms

of inference with speed of physical constraint of the CPU as the limiting measure.

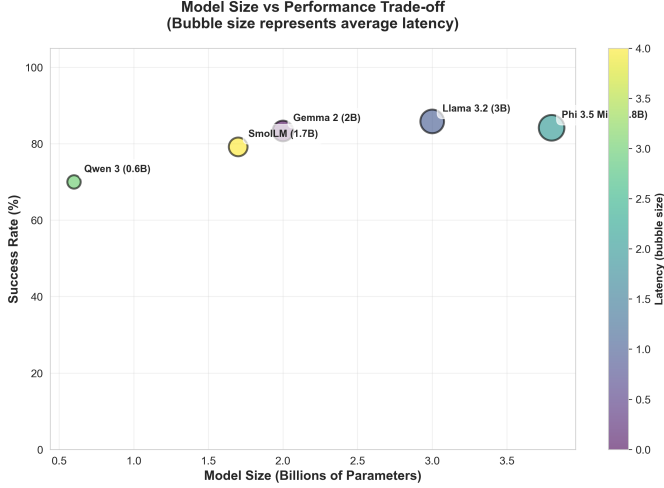


Fig. 1. Performance Pareto Frontier: Model Size vs. Success Rate (Bubble size represents latency).

The performance Pareto Frontier of the bubble sizes that are used to determine the model size is well depicted in figure 1 (Performance Pareto Frontier) of trade-off between the reasoning depth and the speed of the model. Though the maximum accuracy is Llama 3.2, the mean latency of Llama 3.2 is 554ms as compared to 500ms, which is usually considered as fluid, real-time user experience. Qwen 3 on the contrary was super-fast, with a mean latency of about 178ms which is an almost instantaneous response but its lower reliability eliminates it in the case of a complex task. The statistics show that Gemma 2 (2.0B) has the best middle-ground with the constant latency profile with 421ms as the principle value and its success rate of the best model is 2.5 percent. It is the 2-billion parameter scale that is the current sweet spot in respect to inference that is founded on the CPU in a mobile setting. Figure 2 Latency Boxplot, also indicates that the larger models were far more diverse, with the Qwen and Gemma indicating that the span in their performance could be predicted, i.e. the model with more than 3.5 parameters cannot be run on mid range mobile processors. A new generation of mobile-centric benchmarks forms the basis of the importance of development of these performance characteristics.

An example of such functional difference that transpires across all the architectures is de-structuring of the success

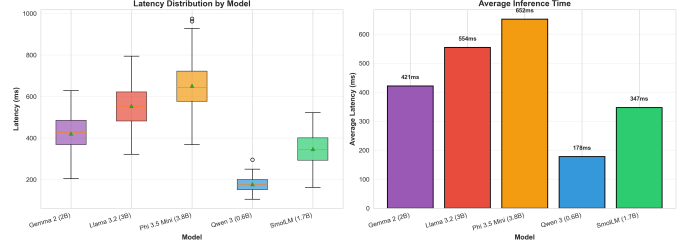


Fig. 2. Latency Boxplots illustrating variance in inference time across architectures.

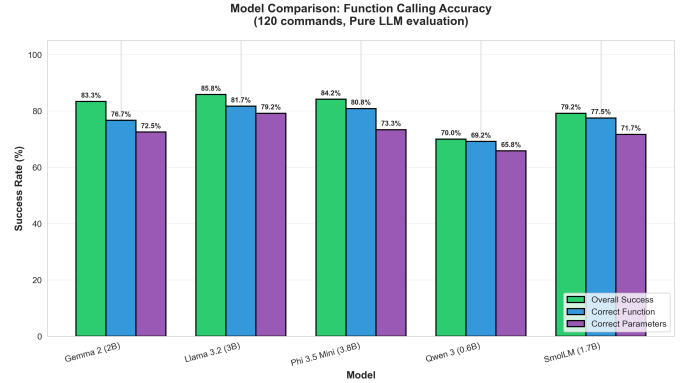


Fig. 3. Component Accuracy Bar Chart: Comparison of Function Accuracy vs. Parameter Accuracy.

rates to component measurements. The specific findings of the analysis that the accrual of the Function Accuracy (choosing the correct API tool) was lower than the accrual of the Parameter Accuracy (obtaining the correct numerical value) are presented in Table I and in the Component Accuracy Bar Chart (Figure 3). One such case was the fact that Llama 3.2 was correct in determining the role in 81.7 per cent of the cases but had a percentage of 79.2 per cent of the correct parameters. The 4-bit quantization, according to this tendency, will not lose the semantic meaning needed to do what to do, it will lose the resolution needed to do how much to do. This means that the quantization noise will have an impact on whether the model will be able to react to certain tokens of number as opposed to the overall intent classification. The result is empirical evidence in the belief that the quality of a model in succeeding in calling a function is a unique quality rather than the quality to accurately provide all the required arguments which is important in the advance of dependable mobile agents.

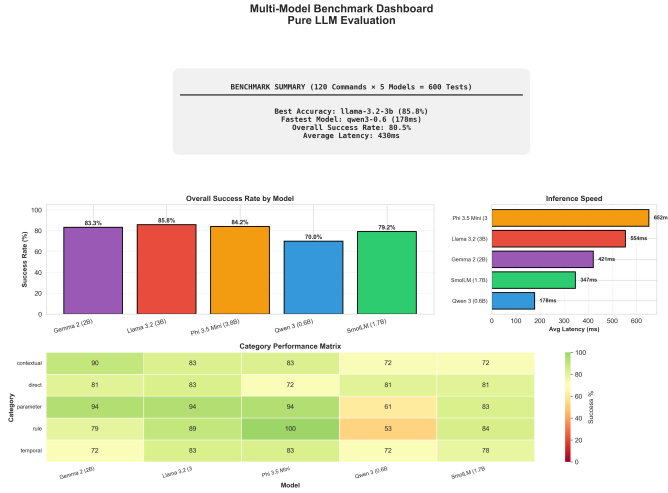


Fig. 4. Category Performance Heatmap showing success rates across distinct command complexities.

This weakness also is revealed in the Figure of the Category Performance Heatmap (Figure 4). Although models have been highly successful in Parameter and Contextual category, Temporal category was a universal antagonist. Most of the models were having troubles in converting relative time words like, every 90 minutes, into absolute integers of the system and the success rate of doing so in this sector was 72-78 percent. That is to say, the linguistic reasoning is incomparable to the symbolic and arithmetic reasoning abilities of the quantized model. Interestingly, the Phi 3.5 model has 100 percent in Rule category indicating that the design is quite a special one with respect to the conditional logic which was scored low in Qwen 3 which is fairly small. The effective implementation of these command proves the ability of the agent in automating the tasks, which is investigated in systems that use the power of LLM reasoning to adhere to a sequence of complex tasks in the Android operating system [23]. The differences in performance observed in the categories indicate that not every agentic task is equal and the model architecture can greatly influence a certain type of reasoning.

A key factor in this study was the experiment of speech recognition false alarms robustness. Among the interesting and counter-intuitive findings, Word Error Rate (WER) presented in the Correlation Matrix (Figure 5) is near-zero related to Success Rate ($r = -0.115$). It implies that even when the quality of the speech transcription is decreased, the system does not lose its power to act on the right command in a linear way.

Semantic probability of the SLM relative to the traditional slot-filling systems that assume fatal error, has the capacity to contextually repair errors. Regarding the case of the ASR writes, once the rights are activated, the SLM can provide an inference of the most probable meaning of the text, basing on the tools to perform a specific action. This result suggests that, in the next generation of mobile agents one will be able to execute better small scale ASR models such as Whisper Tiny without the need to compromise the overall system

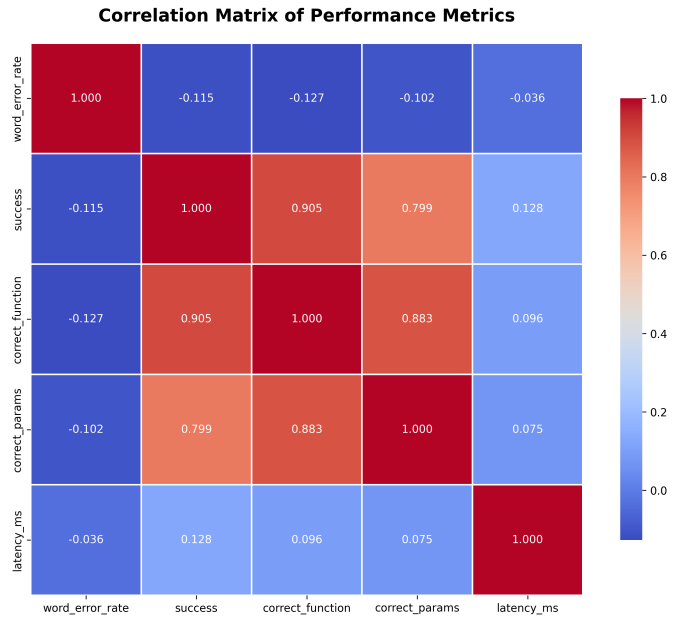


Fig. 5. Correlation Matrix of Performance Metrics.

reliability since the shortcomings of the perceptual layer are compensated by the rationale of the language model. This Semantic Resilience via a function calling agent is a new discovery, unlike other mobile agent systems which are powered by visual interpretation like which uses GUI screenshot interpretation in place of logical function call interpretation [24]. All this shows that even in the entirely offline case voice-native automation can be as competitive with cloud-based systems as the large parameters of the model would permit that the tradeoff between the smartness of the large parameters and the low-latency of the mobile benefit be made. Although this resilience can be improved further with personalization, which is based on the existing approaches to on-device adaptation. To illustrate, the accuracy gains following the fine-tuning would be a direct application of frameworks that depend on self-supervised data selection and synthesis to tweak a model to the habits and the vocabulary of a particular user without impairing the privacy [25].

V. CONCLUSION

The mobile computing trend is gradually characterized by a move towards independence, privacy and less dependence on centralised infrastructure. This work has served as a bridging work between the theoretical possibility of SLMs and their practical feasibility to be used in real time and on-device voice automation. The research, through the creation and strict benchmarking of a single structure, mediating quantized speech recognition and compact reasoning engines, shows that server level intelligence may be optimally reduced to the resource level of a typical smartphone. The fact that this pipeline was successful proves that the reliance on cloud connectivity as something that was once thought to be a precondition to natural language understanding no longer plays

a role in the architectures but is a design choice that can be avoided with optimized software engineering.

The empirical results of the extensive benchmarking of 5 different model architectures indicate that the way to ubiquitous mobile AI does not consist in increasing the number of parameters, but rather finding the best balance between the reasoning depth and the inference latency. Although the larger models such as the Llama 3.2 show the marginal gains in complex reasoning, the computational cost incurred to them makes them not viable to fluid-interact with existing CPUs. The discovery that the 2-billion parameter scale, namely, the Gemma 2 architecture, will be the operational sweet spot, gives a very clear roadmap to follow in further development. It implies that in the particular field of device management and automation, models do not have to have the encyclopedic understanding of a chatbot; they merely have to be able to gain enough semantic understanding to correlate user intent to organized logic. This observation means that the scale orientation of the industry would have reduced returns to edge applications, where speed and battery efficiency are the key factors to determine user satisfaction.

REFERENCES

- [1] F. Wang, Z. Zhang, X. Zhang, Z. Wu, T. Mo, Q. Lu, W. Wang, R. Li, J. Xu, X. Tang, Q. He, Y. Ma, M. Huang, and S. Wang, "A comprehensive survey of Small Language Models in the era of Large Language Models," arXiv preprint arXiv:2411.03350, 2024.
- [2] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, "A review on Edge Large Language Models: Design, execution, and applications," *ACM Computing Surveys*, vol. 57, no. 8, 2025.
- [3] A. Radford, J. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," arXiv preprint arXiv:2212.04356, 2022.
- [4] L. Ben Allal, A. Lozhkov, E. Bakouch, G. M. Blázquez, G. Penedo, L. Tunstall, A. Marafioti, H. Kydliček, A. P. Lajarin, V. Srivastav, J. Lochner, C. Fahlgren, X.-S. Nguyen, C. Fourier, B. Burtenshaw, H. Larcher, H. Zhao, C. Zakkas, M. Morlon, C. Raffel, L. von Werra, and T. Wolf, "SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model," arXiv preprint arXiv:2502.02737, 2025.
- [5] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra, "MobileLLM: Optimizing Sub-Billion Parameter Language Models for On-Device Use Cases," *ICML*, 2024.
- [6] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv preprint arXiv:2312.00752, 2023.
- [7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," arXiv preprint arXiv:2106.09685, 2021.
- [8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *NeurIPS*, 2023.
- [9] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," arXiv preprint arXiv:2211.17192, 2022.
- [10] Y. Qin, S. Liang, Y. Du, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, Y. Liang, W. Hua, S. Ni, J. Lu, Y. Lin, J. Chen, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs," arXiv preprint arXiv:2307.16789, 2023.
- [11] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large Language Model Connected with Massive APIs," arXiv preprint arXiv:2305.15334, 2023.
- [12] R. Qin, D. Liu, G. Xu, A. Nassereldine, Z. Yan, C. Xu, Y. Hu, S. Wang, X. S. Hu, J. Xiong, and Y. Shi, "Tiny-Align: Bridging Automatic Speech Recognition and Large Language Model on Edge," arXiv preprint arXiv:2401.13766, 2024.
- [13] Z. Wang, R. Murthy, Y.-H. Chen, L. Yang, J. Tan, T. M. Awalgaonkar, Y. Zhou, S. Heinecke, S. Desai, J. Wu, R. Xu, S. Tan, J. Zhang, Z. Liu, S. Kokane, Z. Liu, H. Wang, C. Xiong, and S. Savarese, "MobileAIBench: Benchmarking LLMs and LMMs for On-Device Use Cases," arXiv preprint arXiv:2402.14785, 2024.
- [14] S. Laskaridis, K. Katevas, L. Minto, and H. Haddadi, "MELTing Point: Mobile Evaluation of Language Transformers," arXiv preprint arXiv:2403.12844, 2024.
- [15] X. Liu, H. Wang, J. Wang, J.-G. Lou, H.-R. Wei, Y. Qin, Y. Jiang, Y. Yang, Y. Zang, Z. Zhang, Y. Liang, Z. Wang, R. Wang, S. Wang, Z. Wang, P.-J. Lin, H.-L. Zhang, Y. Liu, Z. Li, Z.-Y. Wu, Y. Wu, Y. Wang, W. Wang, X. Wan, X. Li, W. Zhang, J. Tang, and Y. Tang, "AgentBench: Evaluating LLMs as Agents," arXiv preprint arXiv:2308.03688, 2023.
- [16] Gemma Team, "Gemma 3 Technical Report," Google DeepMind, 2025.
- [17] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A Survey on Model Compression for Large Language Models," *Trans. Assoc. Comput. Linguistics*, 2024.
- [18] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," arXiv preprint arXiv:2210.17323, 2022.
- [19] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," *MLSys*, 2024.
- [20] K. Alizadeh, I. Mirzadeh, D. Belenko, S. K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a Flash: Efficient Large Language Model Inference with Limited Memory," arXiv preprint arXiv:2312.11514, 2023.
- [21] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast Large Language Model Serving with a Consumer-Grade GPU," arXiv preprint arXiv:2312.12456, 2023.
- [22] D. Xu, W. Yin, X. Jin, Y. Zhang, S. Wei, M. Xu, and X. Liu, "LLMCad: Fast and Scalable On-Device Large Language Model Inference," arXiv preprint arXiv:2309.04255, 2023.
- [23] H. Wen, Y. Li, G. Liu, S. Zhao, T. Yu, T. J.-J. Li, S. Jiang, Y. Liu, Y. Zhang, and Y. Liu, "AutoDroid: LLM-powered Task Automation in Android," arXiv preprint arXiv:2308.15272, 2024.
- [24] J. Wang, Y. Huang, W. Lei, P. Jiang, H. Liu, C. Wang, F. Wang, and Z. Chen, "Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception," arXiv preprint arXiv:2401.16158, 2024.
- [25] R. Qin, J. Xia, Z. Jia, M. Jiang, A. Abbasi, P. Zhou, J. Hu, and Y. Shi, "Enabling On-Device Large Language Model Personalization with Self-Supervised Data Selection and Synthesis," arXiv preprint arXiv:2311.12275, 2023.